

Adventures in Machine Translation using Python and LLMs

Damir Korenčić
Laboratory for Machine Learning
and Knowledge Representation
Ruđer Bošković Institute

May 12, 2026

Machine Translation – Very Brief History

- The first NLP task
- An important driver of NLP progress
- 1950s-1990s: Rule-based Systems
- 1990s-2010s: Statistical MT
- 2014-2016: Neural Machine Translation
 - RNN-based encoder-decoder models
 - Significant performance increase
- 2017- The Transformer Era
 - "Attention is All You Need" paper (Vaswani et al., 2017)
 - Massive multilingual models (mBERT, mT5, NLLB)
 - Automated **MT quality metrics**: COMET, MetricX, ...
- 2023- The LLM Era ...

**Findings of the WMT24 General Machine Translation Shared Task:
The LLM Era is Here but MT is Not Solved Yet**

Tom Kocmi
Microsoft

Eleftherios Avramidis
DFKI

Rachel Bawden
Inria, Paris, France

Ondřej Bojar
Charles University

Anton Dvorkovich
Dubformer

Christian Federmann
Microsoft

Mark Fishel
University of Tartu

Markus Freitag
Google

Thamme Gowda
Microsoft

Roman Grundkiewicz
Microsoft

Barry Haddow
University of Edinburgh

Marzena Karpinska
UMass Amherst

Philipp Koehn
Johns Hopkins University

Benjamin Marie
The Kaichup

Christof Monz
University of Amsterdam

Kenton Murray
JHU

Masaaki Nagata
NTT

Martin Popel
Charles University

Maja Popović
DCU & IU

Mariya Shmatova
Dubformer

Steinþór Steingrímsson
The Árni Magnússon Institute

Vilém Zouhar
ETH Zürich

WMT24: State of the Field

- LLMs dominate, reaching human level
 - Humans win 5/11 language pairs
 - Humans in the winning cluster 8/11 language pairs
- The best system: Claude3.5-Sonnet
- The best participating system: UnbabelTower-70B
LLama3-70B tuned for MT
- Smaller models can be competitive: IOL-Research-21B

- **31 language pairs:** High- and low-resource languages
 - “General” subtask – 16 pairs
Arabic (Egyptian), Bhojpuri, Chinese, Czech, Estonian, German, Icelandic, Japanese, Maasai, Russian, Serbian (Cyrillic script), Ukrainian
 - “Multilingual” subtask – 15 pairs
Bengali, Greek, Hindi, Indonesian, Italian, Kannada, Lithuanian, Marathi, Persian, Romanian, Serbian (Latin script), Swedish, Thai, Turkish, Vietnamese
- **6 domains:** news, social, speech, literary, educational, dialogue
- English→Target pairs dominate

Motivation

- Working with the technology
- Learning about MT
 - Translation of English LLM benchmarks
 - Adaptation (?)

IRB-MT at WMT25 Translation Task: A Simple Agentic System Using an Off-the-Shelf LLM

Ivan Grubišić*

Division of Electronics
Ruđer Bošković Institute
Zagreb, Croatia

Damir Korenčić*

Division of Electronics
Ruđer Bošković Institute
Zagreb, Croatia

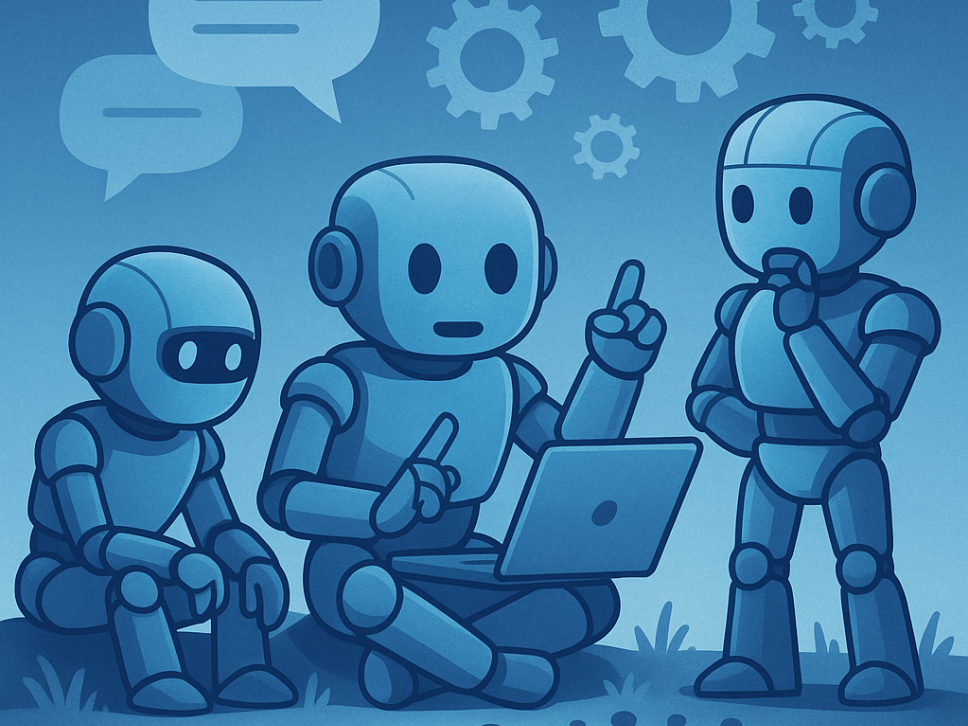
IRB-MT at WMT25 Terminology Translation Task: Metric-guided Multi-agent Approach

Ivan Grubišić*

Division of Electronics
Ruđer Bošković Institute
Zagreb, Croatia

Damir Korenčić*

Division of Electronics
Ruđer Bošković Institute
Zagreb, Croatia



IRB-MT System: Efficient “Off-the-Shelf” Approach

- No post-training (off-the-shelf model)
- Multilingual LLMs of small size (inference-efficient)
- Simple workflow (self-refine)

“Constrained” Track of WMT

- Openly available datasets and models
- Models size $< \mathbf{20B}$ **parameters**
- Fair comparison of resource-efficient approaches

Stage 1: Translate

Source Text

Translation Prompt

Translate
(temp=0)

Initial Translation

Stage 2: Self-Refine

Source Text

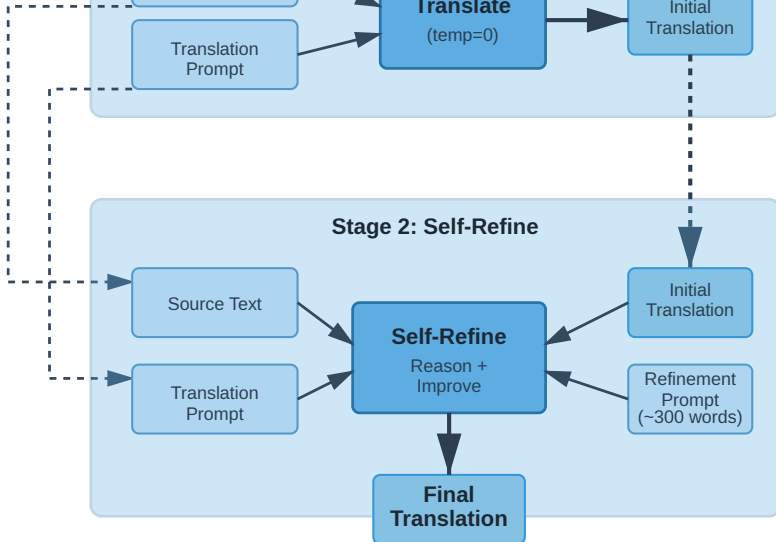
Translation Prompt

Self-Refine
Reason + Improve

Initial Translation

Refinement Prompt
(~300 words)

Final Translation



Model Selection

- Gemma3-12B, Qwen3-8B, EuroLLM-9B, Aya-101 (13B), NLLB-200 (3.3B)
- Evaluation: MetricX, WMT24++ dataset

Model	Qwen3	NLLB	Gemma3	EuroLLM	Aya
MetricX	6.98	6.54	4.59	5.03	5.01
Std. Dev.	3.40	3.70	2.59	2.83	2.88

Translation

- Gemma3-12B for translation & refinement
- Paragraph-level translation (no document context)
- Text-only input (ignores multimodal data)
- Temperature = 0

Implementation & Execution

- LLM models (LangChain, Hugging Face)
- Evaluation (MetricX-24)
- Translation workflow
- Configuration
- Running the workflow (MN5 GPU cluster)

LLM models

- LangChain
 - Unified interface (adapters)
 - **Caching**
- Hugging Face
 - Unified interface (to models' implementation)
 - Model management (repository, saving/loading, ...)
 - Inference endpoints

LLM models – full stack

- Translator
- LangChain
- Hugging Face
- PyTorch/TensorFlow
- CUDA/C++/C
- GPU/CPU

LLM models – factories

- Generic factory methods
 1. Configure HF model (Pipeline)
 2. Wrap in a LC adapter
 - ▶ HuggingFacePipeline
 - ▶ ChatHuggingFace
 - `create_generative_chat_model()`
 - `create_generative_llm_model()`
 - `create_seq2seq_translate_llm_model()`
- Model-specific factory methods
 - `create_eurollm_chat_model()`
 - `create_qwen3_llm_model()`
 - `create_aya_translation_model()`
 - default parameters – full configuration

LLM models – caching

- Key for speeding-up the experiments
- LangChain caching
 - SQLite cache
 - key: {model parameters, model input}
 - `_identifying_params` field
 - automatic

```
1 def turn_on_langchain_caching():
2     global _LANGCHAIN_CACHING_ENABLED
3     if not _LANGCHAIN_CACHING_ENABLED:
4         set_llm_cache(SQLiteCache())
5         _LANGCHAIN_CACHING_ENABLED = True
```

LLM models – caching

```
1 @enhance_identifying_params(include_kwargs=['model_id',  
  ↳ 'temperature', 'top_p', 'top_k', 'torch_dtype',  
  ↳ 'max_new_tokens'])  
2 def create_generative_chat_model(  
3     model_id: str,  
4     device_map: Union[str, Dict[str, Any]] = "auto",  
5     max_memory: Optional[Dict[int, str]] = None,  
6     torch_dtype: Optional[torch.dtype] = None,  
7     temperature: float = 10e-10,  
8     max_new_tokens: int = 1024,  
9     top_k: int = 50,  
10    top_p: float = 1.0,  
11    **kwargs  
12 ) -> ChatHuggingFace  
13     ...
```

Evaluation

- Model selection
- Final system (sanity) check
- MetricX-24
 - Based on google/mt5-xxl
 - Trained on annotated translation data
- Input
 - Original text
 - Translation
 - Reference text
- Output $\in [0, 25]$
- `MetricXTranslationMetric(ITranslationMetric)`

Translation

- Translation
 - Translator
 - ▶ `def translate(self, text: str, **kwargs)`
 - ▶ prepare input
 - ▶ query the model
 - ▶ extract translation
 - TranslationStrategy
- Workflow (multi-LLM)
 - The “custom code” approach
 - Implements the Translator

Configuration

- “Experiment run” parameters
 - agents
 - datasets
 - metrics
 - max_examples
 - use_sampling
- YAML config file
- CLI param override

Platform

- EuroHPC "Development" Grant
- MareNostrum MN5 GPU Cluster
- Environment
 - `module load cuda/12.6 python/3.12.1-gcc`
 - `setup venv (pip install from a local folder)`
- SLURM scheduling system
- `run.slurm`
 - `define resources (1 node, 1 gpu)`
 - `srun python translate.py`
- `run_slurms.sh`
- AI Factories: "aim to offer advanced computing resources and expert support (MINERVA) to European industry and scientific communities." – "Fast Lane" access

Dataset

- “General MT” subset
- English to: Arabic, Bhojpuri, Chinese, Czech, Estonian, Icelandic, Italian, Japanese, Korean, Maasai, Russian, Serbian (Latin), Ukrainian; Czech-to-German, Czech-to-Ukrainian, and Japanese-to-Chinese
- “Multilingual” subset
- English to: Bengali, German, Greek, Persian, Hindi, Indonesian, Italian, Kannada, Lithuanian, Marathi, Romanian, Serbian (Cyrillic), Swedish, Thai, Turkish, Vietnamese
- Six domains: News, Literary, Speech, Social, Education, Dialogue
- Cca. 70 to 350 words per text
- On average, 1-4 paragraphs per text
- Cca. 100 to several 1,000's texts per language

Results: Not Great, Not Terrible

- Mid-tier translation system
- Model post-training leads to better performance
- Top non-post-trained system
(including the other agentic systems)
- Large language-dependent variance
- Good performance: German, Chinese, Russian
Ukrainian, Serbian
- Competitive performance: Arabic
- Weak performance: Estonian, Icelandic, Japanese-Chinese

Summary

- Self-refine works for MT (outperforms Gemma3-12B-IT)
- Reasonably good and **resource-efficient** baseline (especially for Slavic languages)
- Performance is highly language-dependent
- Room for improvement
 - Post-training (fine-tuning, ...)
 - More complex workflows
 - Inference efficiency: optimized software, batch processing

WMT25: State of the Field

- LLMs dominate, **surpassing human level**
 - Humans in the winning cluster for 6/16 language pairs
 - Benchmarks need to become harder
- The best system: Gemini 2.5 Pro
- **Small-LLM-systems nearing state-of-the-art**
 - **Hunyuan-MT** (Tencent):
Hunyuan-MT-7B, Hunyuan-MT-Chimera-7B
 - Marco-MT-Alghrab (Alibaba):
adaptation of Qwen3-14B
- Findings of the WMT25 General Machine Translation Shared Task: <https://aclanthology.org/2025.wmt-1.22.pdf>

WMT26 Shared Task

- <https://www2.statmt.org/wmt26/translation-task.html>
- Domains
 - Instruction following
 - ▶ Formal/informal voice
 - ▶ Glossaries
 - ▶ Structured (format) translation
 - ▶ Style and expressions
 - Spoken dialogue
 - Social
- Constrained/Unconstrained tracks

Takeaways

- Evaluation is key
 - Realistic performance, avoid over-engineering
 - Open datasets
 - LLM-as-a-judge
 - Synthetic evaluation data?

A recipe for a MT system

- Prompting of reliable LLMs – a good baseline
- Multi-LLM workflow expectedly helps
- Post-training (if you have data and compute)
- Hugging Face Inference

The end...

- '25 Shared Task Data:
<https://github.com/wmt-conference/wmt25-general-mt>
- Our systems: <https://github.com/igrubi/irb-mt-wmt2025>
- Self-refine GUI demo:
<https://github.com/dkorenci/llm-mt-demo>

